# NATIONAL ENGINEERING COLLEGE, KOVILPATTI

Department of Computer Science and Engineering

## 23CS55C– ARTIFICIAL INTELLIGENCE
## MICRO PROJECT REPORT

# NEC Fee Assist: AI Task and Navigation System

(Learned Task Classification and NLP-Based Fee Assistance)

**Submitted by**

**SYED SUFYAN 2403957610421048**

**Course Instructor**

| Innovation and Problem statement (10 Marks) | Implementation & Results (10 Marks) | Presentation and Documentation (10 Marks) | Viva (10 Marks) | Total (40 Marks) |
|---|---|---|---|---|
| | | | | |

# TABLE OF CONTENTS


| CH NO | TITLE | PAGE NO |
|---|---|---|
| 1 | INTRODUCTION | 3 |
| 2 | PROBLEM STATEMENT | 5 |
| 3 | MODULE DESCRIPTION | 6 |
| 4 | ARCHITECTURE DIAGRAM | 9 |
| 5 | CODING & IMPLEMENTATION | 12 |
| 6 | OUTPUT SCREENSHOTS | 20 |
| 7 | CONCLUSION | 24 |

# CHAPTER 1
# INTRODUCTION

College fee portals contain assigned fees, due dates, payment attempts and receipts, but students often need to visit several screens to complete one task. A student may know what they want to do without knowing the portal terminology or where the relevant control is located. Questions such as "How much do I owe?" and requests such as "Help me pay tuition" require different responses: the first needs account information, while the second needs a guided workflow.

This micro project presents NEC Fee Assist, an AI-based student fee assistance and task navigation system. It combines an authenticated student/admin portal, an offline language engine and a bounded task agent. The assistant can answer account questions, retrieve maintained fee policies, open relevant portal pages, select the requested pending fees, find completed-payment receipts and guide users through the next steps.

## The Gap:

A question-answer chatbot can return useful text but still leave the student searching for the correct screen. A purely scripted command list also handles only a narrow set of phrases. At the same time, giving unrestricted financial authority to natural language input would make mistakes difficult to control. The need is therefore for an assistant that recognizes task requests, takes permitted interface actions and keeps financial confirmation in the normal portal workflow.

## The Goal:

The primary objective is to build an accessible, explainable AI assistant that connects language understanding with practical portal tasks. The system uses a learned multinomial Naive Bayes task classifier trained from labelled examples, TF-IDF cosine evidence, authenticated account retrieval and a fixed registry of interface tools. Key functionalities include:

- Student and administrator sign-in with separate roles and account-scoped records.
- Source-grounded answers for balances, fee deadlines, payment history and maintained policies.

- Automatic navigation to permitted portal pages from natural language task requests.
- Preparation of a category-specific payment by selecting owned pending fees for review.
- A persistent assistant panel that remains available on the destination page.
- Session-scoped task guidance, continuation and cancellation, with fresh checks of payment state.
- Protected receipt links for completed simulated payments and guided administrator workflows.

The implementation is developed using Python, SQLite, HTML, CSS and JavaScript. The learned task model is implemented directly with sparse feature counts, Laplace smoothing and log likelihoods. Policy search uses unigram/bigram TF-IDF and cosine similarity. The assistant operates locally without an external language-model service or API key.

## Project Scope:

The project is an independent academic demonstration. Its SQLite database, illustrative fee amounts and simulated payment outcomes are separate from the official college ERP and the production NEC fee portal. No money is transferred. The assistant prepares and guides tasks; the student completes a payment only through the existing demo payment confirmation form. Administrator changes likewise use the normal page forms.

## AI Contribution:

The task classifier learns word and phrase likelihoods from 80 labelled examples across 12 classes, including a class for information questions. It does not consist only of one rule for every task phrase. Request detection and role checks constrain when a learned prediction may become an action. TF-IDF evidence and a class-separation threshold allow the planner to abstain from unsupported commands.

The project demonstrates language representation, probabilistic classification, retrieval, task planning, tool validation and evaluation in one workflow. It is a bounded classical AI agent rather than a generative large language model. Its vocabulary and tool set are deliberately limited and its measurements are reported as local development diagnostics.

# CHAPTER 2
# PROBLEM STATEMENT

The core problem is the absence of a conversational layer that helps students both understand fee information and reach the controls needed to complete a task. A conventional portal displays records but expects the student to understand its menu structure. A text-only assistant may explain the next step without taking the student to the appropriate page or preparing the relevant fee selection.

This deficiency manifests in several critical areas:

- Fragmented Navigation: Students must move between overview, pending fees, transaction history and profile pages to resolve related questions and tasks.
- Variation in Language: Requests such as "Open my profile", "Take me to account details" and "Could you navigate to my profile?" should identify the same destination without a separate exact-match command for each sentence.
- Account Grounding: Balances, due dates and receipt availability must come from the signed-in account rather than from generic policy text or a guessed answer.
- Task Continuity: A useful assistant must retain the current task, explain what comes next and check whether the selected fees are still unpaid before continuing.
- Authorization and Financial Control: A student request must not open administrator functions, access another student's records or bypass payment confirmation.

The proposed solution is an offline AI-assisted fee portal with a learned task classifier and a controlled execution layer. The assistant selects from predefined interface tools, applies the authenticated role and owner checks, and provides a clear fallback when it cannot support a request. The application retains ordinary confirmation forms for payments and administrator changes, while the conversational layer handles navigation, selection and guidance.

# CHAPTER 3
# MODULE DESCRIPTION

NEC Fee Assist is structured into seven key modules. Together they convert a signed-in user's question or task request into a grounded answer or an authorized interface action.

- Authentication and Access Control Module: Uses salted PBKDF2 password hashes, expiring server-side sessions, HttpOnly/SameSite cookies, CSRF validation and separate student/admin roles. The server derives the actor from the session.
- Student Fee Management Module: Displays fee allocations, amounts and due dates, supports pending-fee selection and records completed, failed or cancelled simulated payments. Receipt access is restricted to the owner or an administrator.
- NLP Answer and Policy Retrieval Module: Resolves account intents and fee categories, reads owned structured records and ranks maintained policy documents with unigram/bigram TF-IDF and cosine similarity.
- Learned Task Understanding Module: Trains a multinomial Naive Bayes classifier from labelled utterances, computes class log likelihoods and uses cosine evidence plus a separation threshold before accepting an interface task.
- Agent Planning and Tool Control Module: Produces only allowed navigation, fee-selection and selection-clear actions. It verifies roles and account data, prepares workflow steps and keeps task state within the authenticated session.
- Conversational Interface and Auto-Navigation Module: Presents answers, source labels, task steps and receipt links. It opens the selected page, keeps the assistant visible and independently validates received action types and fee identifiers.
- Administration and Evaluation Module: Provides student creation, fee allotment, ledger export, policy maintenance and audit records, together with isolated automated tests, browser checks and authored diagnostic evaluation.

## Architectural Foundation:

The browser communicates with a Python HTTP service using authenticated JSON endpoints. Application services enforce the session, role and account boundary before reading or changing records. The language engine and task agent run in the same local Python process, while SQLite stores account, fee, policy and conversation data. This modular structure permits the task model or retrieval logic to be changed without rewriting the portal screens.

## Functional Modules:

| Project file | Responsibility |
|---|---|
| app/server.py | HTTP routes, cookie handling, host/origin validation, request bounds and CSRF checks. |
| app/service.py | Authentication, authorized fee services, chatbot orchestration, agent state and receipt access. |
| app/nlp.py | Tokenization, unigram/bigram features, TF-IDF ranking, account intents and fee categories. |
| app/agent.py | Labelled training examples, learned task classification, bounded plans and role-scoped tools. |
| app/schema.sql | Relational schema, ownership keys, constraints and stored conversation/action fields. |
| app/static/app.js | Portal pages, assistant panel, automatic page changes and independent action validation. |
| app/static/style.css | Responsive portal and assistant layout. |
| tests/test_project.py | Temporary-database service, HTTP, payment and agent regression checks. |
| scripts/evaluate_ai_agent.py | Authored task and safety diagnostic with measured results. |

## Operational Flow:

The user signs in and opens Fee assistant from the navigation or the floating launcher. The user submits a question or task. The server reloads the current session and account records, then chooses the task-planning path or the ordinary account/policy answer path.

For a task request, the agent predicts a task class and checks its relevance before constructing an allowed action list. "Help me pay tuition" produces navigation to Pending fees and a selection containing only the signed-in student's unpaid tuition allocations. The returned guide explains review, payment confirmation and receipt access. The browser executes the permitted interface actions and retains the assistant alongside the destination page.

For a question, the answer service reads the appropriate account records or retrieves the highest-ranked maintained policy. Account questions never use text similarity to invent a balance or due date. The response includes its source, and the exchange is recorded in conversation history.

When the user asks for the next step, the server reads the task from the current session. A payment guide rechecks whether its fees remain unpaid. If they have already been settled, the assistant opens Transaction history instead of selecting them again. Cancellation clears the task state and the browser fee selection.

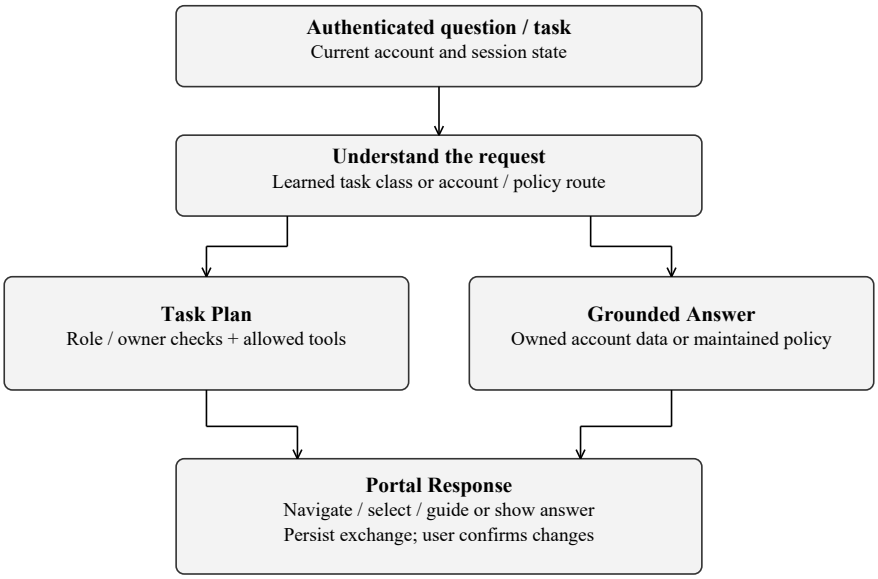



Figure 3.1: End-to-end question and task execution pipeline.

All financial and administrator writes remain ordinary authenticated form submissions. Neither model predictions nor stored history can execute a payment. Reopening conversation history displays prior actions as data without replaying them.

# CHAPTER 4
# ARCHITECTURE DIAGRAM

## 4.1 Overview:

The architecture separates presentation, the authenticated HTTP boundary, language understanding, bounded task planning and persistent data. The browser remains responsible for interface navigation, while the server controls the identity, permitted destinations and owned fee identifiers supplied to each plan. The model cannot construct arbitrary URLs, SQL or executable scripts.

## 4.2 Architectural Diagram:

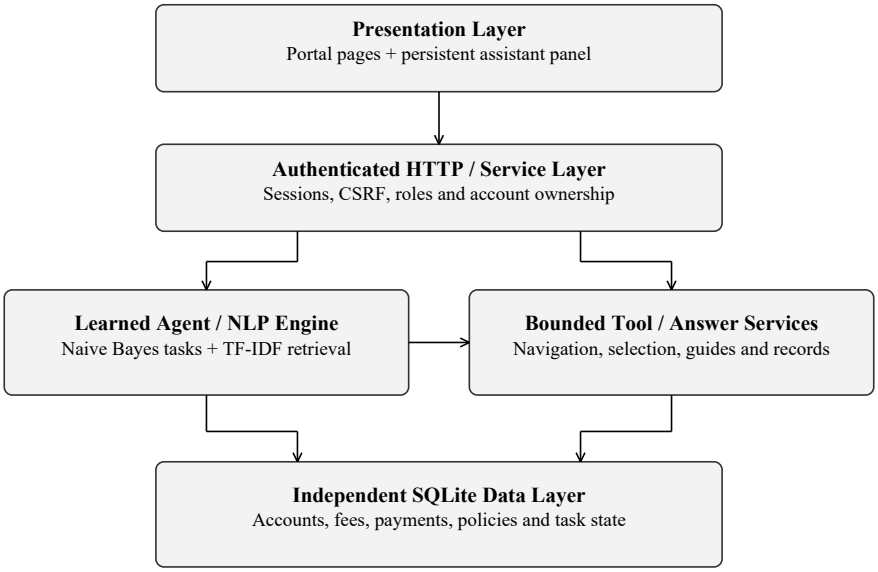



Figure 4.1: Layered architecture of NEC Fee Assist.

## 4.3 Layer Descriptions

### 4.3.1 User Interface Layer

- Component: Responsive HTML, CSS and JavaScript student/admin portal.
- Functionality: Overview, fees, history, profile, administrator pages and a persistent assistant panel.
- Purpose: Connect natural language assistance with visible portal controls.

### 4.3.2 Application Layer

- Component: Python HTTP routes and application services.
- Functionality: Authentication, session and CSRF checks, role/ownership validation, request limits, account retrieval, payment simulation and protected output downloads.
- Purpose: Provide the controlled boundary between language requests and fee records.

### 4.3.3 AI and NLP Processing Layer

- Component: app/agent.py and app/nlp.py using Python sparse dictionaries, counters, regular expressions and mathematical functions.
- Functionality: Feature extraction, learned multinomial Naive Bayes classification, cosine evidence, account-intent resolution and maintained-policy retrieval.
- Purpose: Choose a supported task or answer source without treating unrestricted text as an executable instruction.

### 4.3.4 Agent Tool and Dialogue Layer

- Component: Role-scoped planner, server-side session task state and browser action executor.
- Functionality: Allowed page navigation, owned fee selection, task continuation, cancellation, guide steps and receipt discovery. The client validates actions again before applying them.
- Purpose: Translate an accepted prediction into a small, reviewable interface plan.

### 4.3.5 Data and Infrastructure Layer

- Component: Independent SQLite database, local runtime directory and loopback-only evaluation server.
- Functionality: Persists users, sessions, fees, simulated payments, settlements, policies, conversation records and audit events. Additive migrations preserve existing project data.
- Purpose: Supply consistent, account-scoped facts and reproducible laboratory execution.

### 4.4 Data Flow

- 1. The browser signs in and receives a server session cookie; subsequent requests use the authenticated account.
- 2. The user submits a question or task through the assistant. The server validates its type and maximum length.
- 3. The server refreshes session identity, task state, owned fees and payment records. Request wording is checked before learned task classification.
- 4. A supported task is converted to allowed navigation and selection actions after role and owner checks. Otherwise, the account/policy answer service handles the question.
- 5. The response carries an answer, source, relevance score, optional actions, guide and protected receipt links. The browser refreshes account data and validates the action list.
- 6. The page opens automatically and requested unpaid fees are selected. The user completes any financial or administrator change in its normal form.
- 7. The conversation and agent task event are stored. A later next-step request rechecks current state; cancellation clears the active task.

### 4.5 Key Design Decisions

- Learned Understanding with Bounded Tools: The model predicts from examples, while a fixed tool registry controls what can happen.
- Account Grounding: Amounts, deadlines and receipts originate from authorized database records.
- User Confirmation: Selecting fees prepares a payment; the existing form remains the confirmation boundary.
- Independent Client Validation: Unknown actions, external destinations and invalid fee identifiers are ignored.
- Transparent Limits: Unsupported requests fall back; the project does not claim unrestricted language understanding or official ERP integration.

# CHAPTER 5
# CODING & IMPLEMENTATION

## 5.1 Introduction

The implementation uses Python for the HTTP service, account operations, learned agent and retrieval engine. The browser interface uses HTML, CSS and JavaScript, and SQLite stores the independent demonstration dataset. ReportLab generates protected demonstration receipts. The core AI calculation requires no external machine-learning service.

## 5.2 Coding

The following extracts show the main implementation boundaries. The source screenshots in Sections 5.5 and 5.6 show these files in the project source viewer.

### 5.2.1 Session and Request Security (server.py):

After sign-in, every API request resolves the server session. State-changing requests require the session CSRF token. Receipt retrieval remains an authorized endpoint even when the agent provides its link.

```
token=self.token(); user=s.session(token)
if method=='POST' and not
   hmac.compare_digest(self.headers.get('X-CSRF-Token',''),user['csrf']): raise
   s.Error(403,'Invalid security token. Refresh and try again.')
if method=='GET' and path=='/api/dashboard': return self.send(200,s.dashboard(user))
if method=='GET' and path=='/api/chat/history': return self.send(200,s.chat_history(user))
if method=='GET' and path.startswith('/api/receipt/'):
   pid=path.removeprefix('/api/receipt/')
   import re
   if not re.fullmatch(r'DEMO-[A-F0-9]{12}',pid): raise s.Error(404,'Receipt not found.')
   return self.send(200,s.receipt(user,pid),'application/pdf',download=pid+'.pdf')
if method=='GET' and path=='/api/admin/export':
   s.require_admin(user); dashboard=s.dashboard(user)
   out=io.StringIO(newline=''); writer=csv.writer(out)
```

### 5.2.2 Password and Session Storage (service.py):

Passwords use salted PBKDF2-HMAC-SHA256 with 600,000 iterations. Only session-token hashes are stored. Cookies are HttpOnly and SameSite=Strict. The local server binds to loopback; production hosting requires a separate HTTPS deployment.

```
def password_hash(password,salt=None):
   salt=salt or secrets.token_hex(16)
   return salt+':'+hashlib.pbkdf2_hmac('sha256',password.encode(),bytes.fromhex(salt),600000).hex()
def password_ok(password,stored): return
   hmac.compare_digest(password_hash(password,stored.split(':')[0]),stored)
```

### 5.2.3 Database and Additive Migration (service.py):

SQLite foreign keys are enabled for each connection. Agent task state and serialized action/guide fields are added when absent, preserving existing student records, fees and payment attempts. New installations declare the same fields in schema.sql.

```
for table,column,definition in [('sessions','agent_state',"TEXT NOT NULL DEFAULT '{}'"),
                                ('chat_messages','actions',"TEXT NOT NULL DEFAULT '[]'"),
                                ('chat_messages','links',"TEXT NOT NULL DEFAULT '[]'"),
                                ('chat_messages','guide',"TEXT NOT NULL DEFAULT 'null'")]:
  if column not in {r['name'] for r in c.execute('PRAGMA table_info('+table+')')}:
    c.execute('ALTER TABLE '+table+' ADD COLUMN '+column+' '+definition)
if c.execute('SELECT COUNT(*) FROM users').fetchone()[0]:
```

### 5.2.4 Text Features and TF-IDF (nlp.py):

The language engine lowercases input, extracts alphanumeric tokens, removes common stop words and constructs unigram/bigram counts. Policy documents and intent examples are ranked with normalized sparse TF-IDF vectors.

```
def features(text):
  ts = tokens(text)
  return Counter(ts + [' '.join(ts[i:i+2]) for i in range(len(ts)-1)])
```

$$tf(t,d) = 1 + \ln(count(t,d))$$
$$idf(t) = \ln((N + 1) / (df(t) + 1)) + 1$$
$$w(t,d) = tf(t,d) * idf(t)$$
$$cosine(q,d) = dot(L2_normalize(q), L2_normalize(d))$$

Policy answers use the selected maintained document only when its cosine score reaches 0.16. The account exemplar route uses a separate 0.28 threshold. These values are implementation settings, not calibrated confidence probabilities. Payment-mode terms can be expanded to the maintained payment-methods topic, improving retrieval while retaining the stored policy as the answer source.

### 5.2.5 Learned Task Classification (agent.py):

The task model is fitted from 80 labelled training examples across 12 classes: fees, history, profile, dashboard, pay, receipt, allotment, students, ledger, policies, audit and answer. Sparse word/phrase counts estimate a multinomial Naive Bayes likelihood with Laplace smoothing and uniform class priors.

```
count(t,c) = feature occurrences in examples of class c
P(t | c) = (count(t,c) + 1) / (total(c) + |V|)
score(c,q) = sum(count(t,q) * ln(P(t | c)))
predicted class = argmax_c score(c,q)
```

```
    bag=nlp.features(question)
    known={t:n for t,n in bag.items() if t in VOCAB}
    if not known:return None
    scores={label:sum(n*math.log((counts[t]+1)/(TOTALS[label]+len(VOCAB)))
                      for t,n in known.items()) for label,counts in COUNTS.items()}
    ordered=sorted(scores,key=scores.get,reverse=True)
    label=ordered[0]
    documents=[];labels=[]
    for target,examples in EXAMPLES.items():
        documents.extend(examples);labels.extend([target]*len(examples))
    ranked=nlp.rank(question,documents)
    evidence=max((score for score,i in ranked if labels[i]==label),default=0)
    margin=(scores[label]-scores[ordered[1]])/max(1,sum(known.values()))
    if label=='answer' or evidence<0.18 or margin<0.035:return None
```

The request detector separates task wording from ordinary information questions. The learned class is accepted only when its matching-example cosine evidence reaches 0.18 and its mean log-score margin reaches 0.035. The answer class and unsupported requests do not execute an interface tool.

The model is deterministic and explainable. Its counts are learned from examples rather than hand-written separately for every command. It remains a small lexical classifier: unfamiliar wording and tasks outside the available portal can still require a fallback. The model does not generate free-form code or arbitrary tool arguments.

### 5.2.6 Bounded Planning and Fee Preparation (agent.py):

A payment task first requires a student role. The planner recognizes the fee category and selects only currently pending allocations from the authenticated account. The resulting actions open Pending fees and populate its checkboxes; they do not submit the payment form.

```
if task=='pay':
    if user['role']!='student':return result('Fee payments are available in a student account.'),state
    categories=[name for name in ['tuition','examination','hostel','transport','library']
                if name in q or (name=='examination' and re.search(r'\bexam\b',q))]
    category=categories[0] if len(categories)==1 else ''
    pending=[f for f in fees if f['status']=='pending' and (not categories or any(name in
        f['head'].lower() for name in categories))]
    if not pending:
        return result('There are no pending '+(category+' ' if category else '')+'fees to
            select.',[navigate('fees')],score=score),state
    ids=[f['id'] for f in pending]
    guide={'title':'Pay '+(category+' fee' if category else 'pending fees'),
           'steps':['Review the selected fees.','Proceed to demo payment.','Confirm the outcome in the
               payment form.','Find the receipt in Transaction history.']}
    answer='Selected '+', '.join(f['head'] for f in pending)+' for review. Confirm the payment in the
        payment form.'
```

### 5.2.7 Browser Action Validation (app.js):

The browser permits only known page identifiers for the current role. Fee identifiers must be safe integers present in the refreshed pending-fee list. Unknown action types have no execution branch. Stored conversation history is rendered without applying its prior action list.

## 5.3 API Endpoints

| Method | Endpoint | Access | Purpose |
|---|---|---|---|
| POST | /api/login | Public | Authenticate and create a session. |
| GET | /api/dashboard | Session | Current authorized account/portal records. |
| POST | /api/chat | Session + CSRF | Question answering or bounded task planning. |
| GET | /api/chat/history | Session | Owned conversation and guide history. |

| Method | Endpoint | Access | Purpose |
|---|---|---|---|
| POST | /api/logout | Session + CSRF | Revoke the current session. |
| POST | /api/payment | Student + CSRF | Confirm a simulated payment attempt. |
| GET | /api/receipt/:id | Owner/admin | Download a completed-payment PDF. |
| POST | /api/admin/fee | Admin + CSRF | Assign an illustrative fee. |
| POST | /api/admin/student | Admin + CSRF | Create a student account. |
| POST | /api/admin/policy | Admin + CSRF | Maintain the policy knowledge base. |
| GET | /api/admin/export | Admin | Download the fee ledger CSV. |

## 5.4 Agent Actions and Task State

| Action or state | Behavior |
|---|---|
| navigate | Open an allowed portal page; no external URL is accepted. |
| select_fees | Select owned pending allocations for review. |
| clear_selection | Clear the browser selection when the task is cancelled. |
| Payment guide | Keep selected fee IDs in the current session; recheck before continuing. |
| Receipt discovery | Return only completed owned payments; the download endpoint checks access again. |
| Administrator guide | Open a permitted form and show its workflow steps. |
| Conversation history | Persist answers, action data, links and guides without replaying actions. |

The assistant does not expose a tool for arbitrary SQL, script execution, external navigation, payment submission or administrator record changes. The user completes those changes through the authenticated portal controls. A request naming another student's register number is not used as a tool argument for student-account actions.

## 5.5 Code Structure

```
NEC FEE BOT/
  app/
    server.py        HTTP routes and security
    service.py       Authorized account and agent services
    agent.py         Learned task model and bounded plans
    nlp.py           Features, account intents and TF-IDF
    schema.sql       SQLite schema and constraints
    static/          HTML, CSS, JavaScript and college assets
  tests/             Service, HTTP and agent regression tests
  scripts/           Evaluation, capture and report generation
  data/              Independent local runtime data
  requirements.txt
  run.ps1
```

## 5.6 Code Screenshots:

The following screenshots display the actual project files and source extracts in the source code viewer.

## (i) Project Files and Database Schema

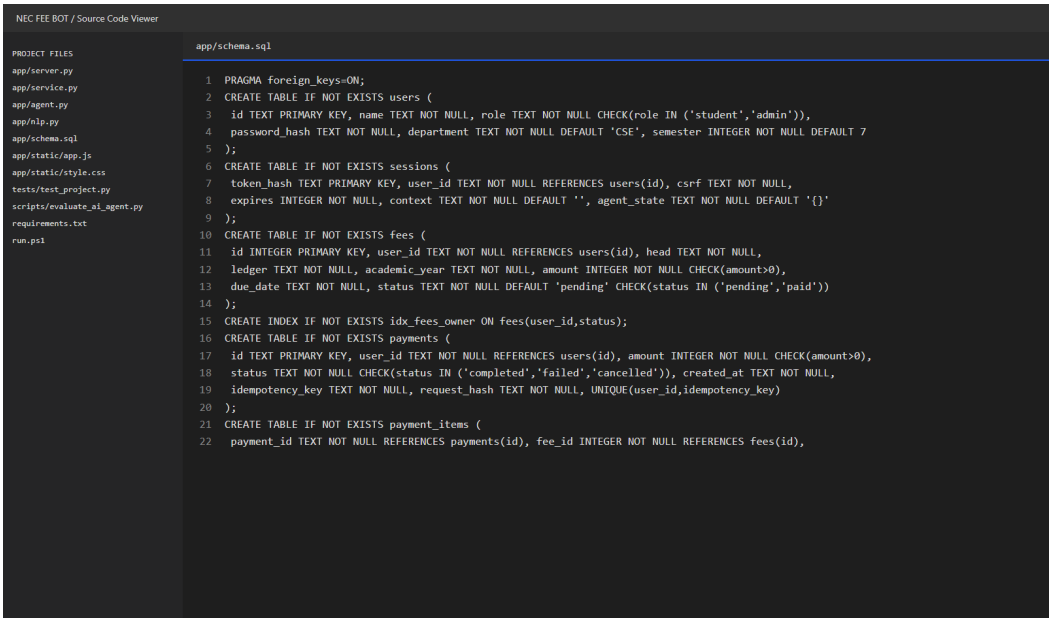



Figure 5.1: Project file tree and current SQLite schema.

## (ii) HTTP Routes and Security Checks (server.py)

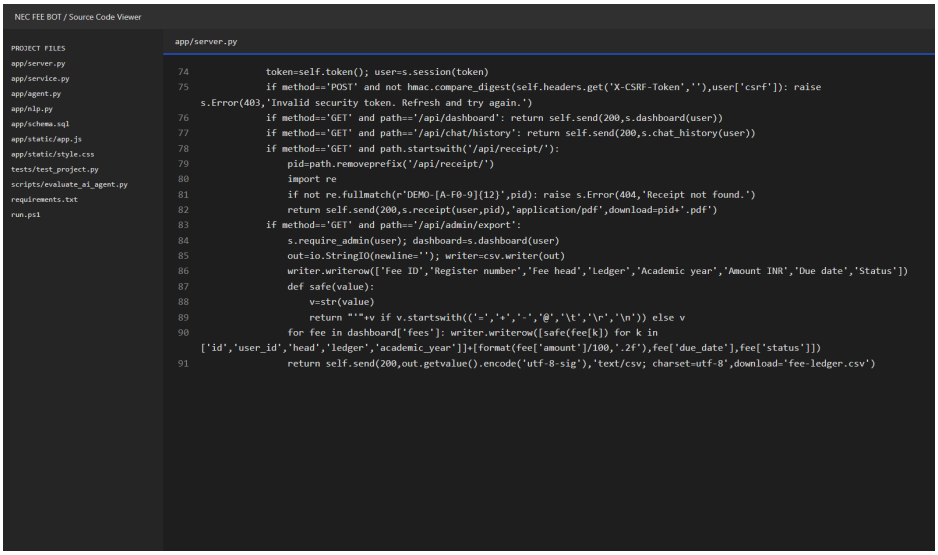



Figure 5.2: Current authenticated HTTP and receipt routes.

## (iii) Learned Task Model and Browser Tools

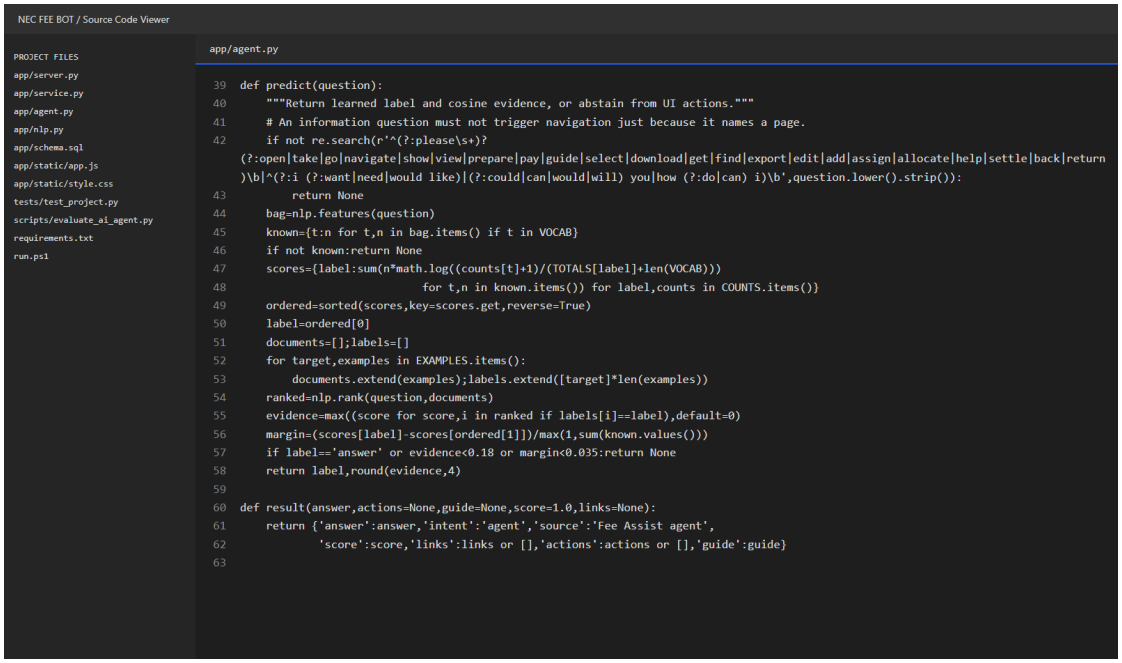



Figure 5.3: Learned task prediction, evidence and acceptance thresholds.

The task model is in app/agent.py. The client tool executor in app/static/app.js separately validates page and fee identifiers before applying the returned actions.

## 5.7 Implementation Details:

- Environment Setup: Install Python, create a virtual environment and install requirements.txt. The offline model uses Python standard-library mathematics and feature counters; ReportLab is required for PDF receipts.

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe -m app.server
```

- Running the Application: Open http://127.0.0.1:8765. On first start, the local demonstration generates random student/admin passwords in data/demo_credentials.json. Existing accounts and fee data are preserved on later starts.
- Agent Operation: Ask a question or request a task. For example, "Could you navigate to my profile?" opens the profile, while "Help me pay tuition" opens Pending fees and selects the owned unpaid tuition allocation. The assistant remains available on the destination page.
- Task Confirmation: Review selected fees and use Proceed to demo payment. The ordinary payment dialog controls the final outcome. Administrator guides similarly stop at the normal form controls.
- Testing: The automated suite contains 43 passing regression tests. The updated browser capture verifies navigation, selection, confirmation, receipt download, role denial, administrator guidance and mobile layout against an isolated temporary dataset.

```
.venv\Scripts\python.exe -m unittest discover -s tests -v
node --check app/static/app.js
.venv\Scripts\python.exe scripts/evaluate_ai_agent.py
```

The browser captures and diagnostic evaluation use separate temporary databases. They do not settle fees or change accounts in the user's regular demonstration database. Runtime credentials and database files are excluded from the source package.

# CHAPTER 6
# OUTPUT SCREENSHOTS

The following screenshots show the updated working application with an isolated academic demonstration dataset.

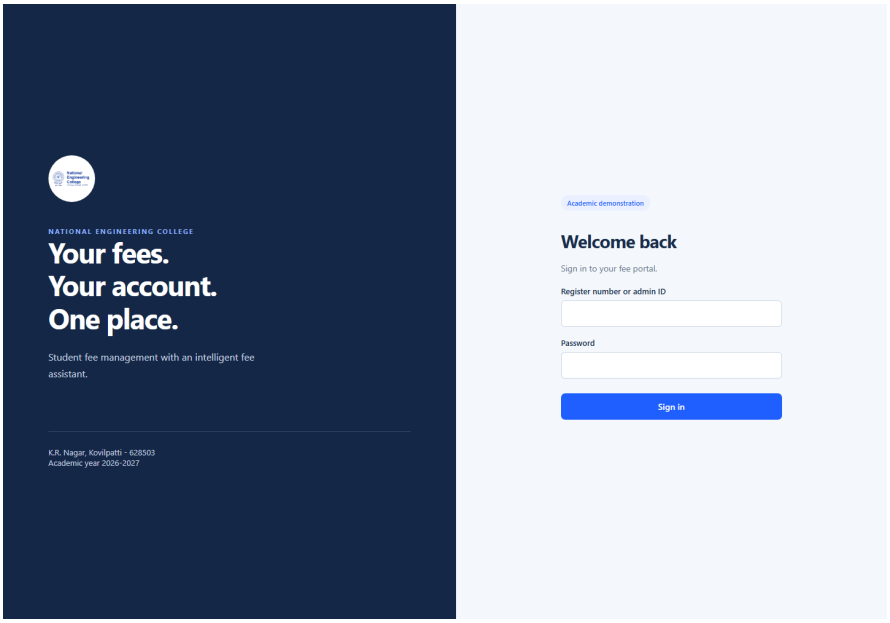



Figure 6.1: Student and administrator sign-in page.

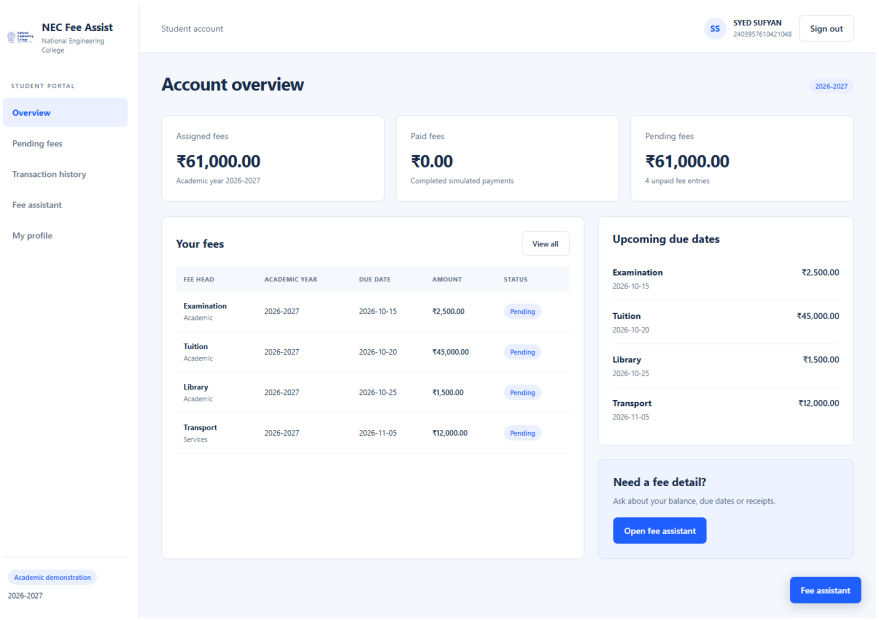



Figure 6.2: Student overview with assigned and pending fees.

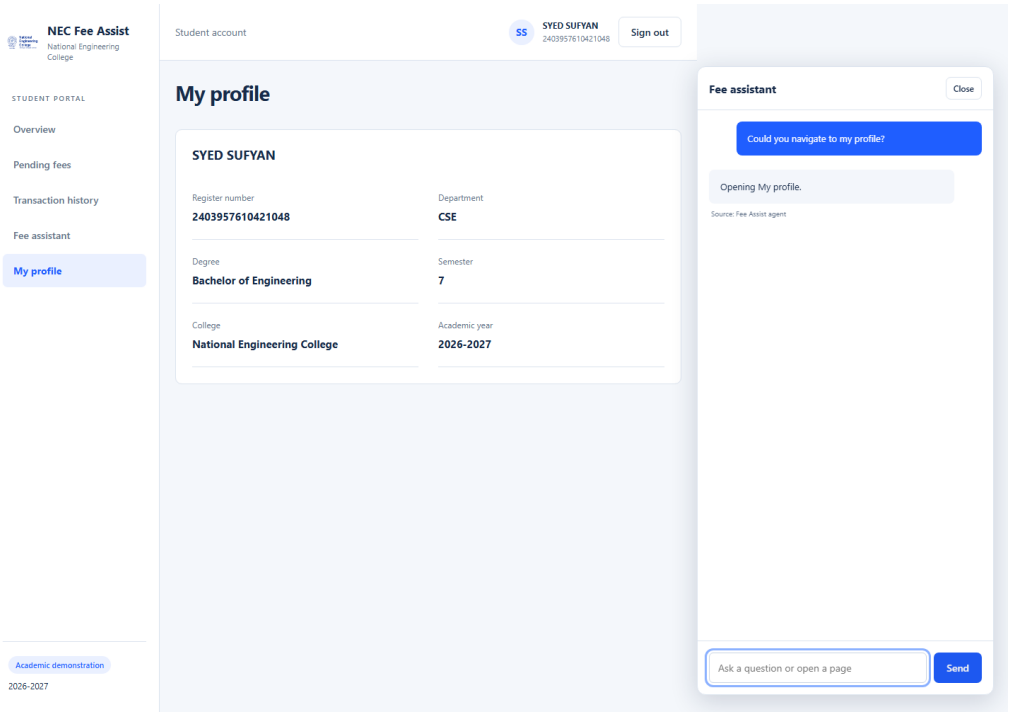



Figure 6.3: Agent automatically opens My profile from a paraphrased request.

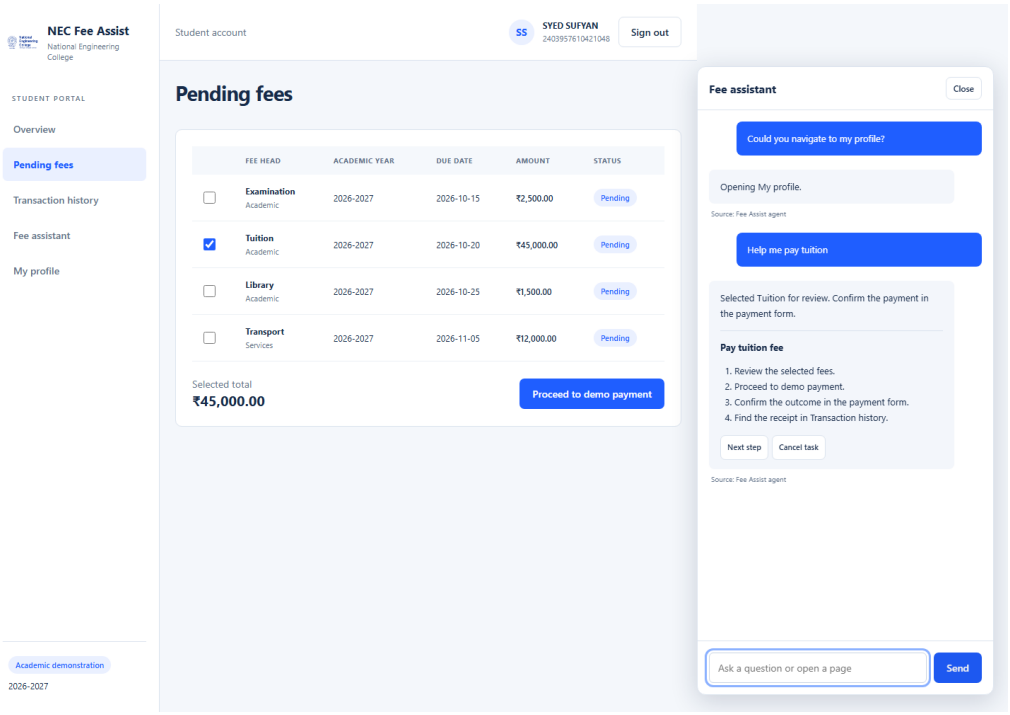



Figure 6.4: Agent opens Pending fees and selects tuition for review.

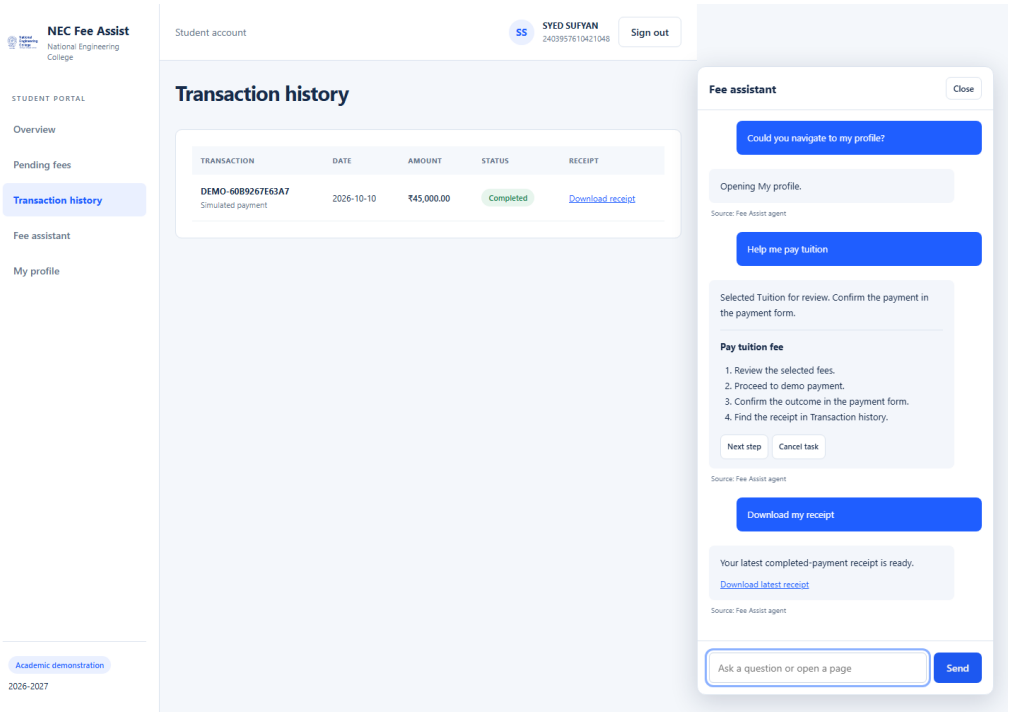



Figure 6.5: Agent finds the latest completed-payment receipt.

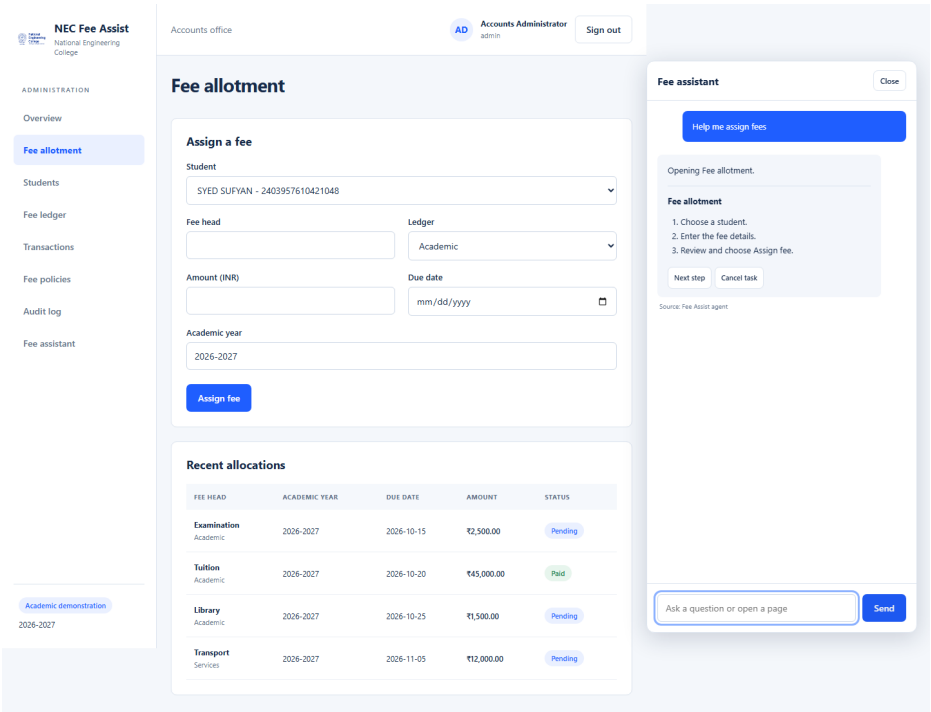



Figure 6.6: Administrator fee-allotment navigation and task guidance.

## 6.1 Model Evaluation Results

The learned task model was checked using 40 authored development requests covering navigation, payment preparation, receipt discovery, information answers, role denial, blocked instructions and cancellation. The phrases are absent verbatim from the 80 training examples, but the diagnostic set was reused during development. It is not an independently collected or untouched test set.

| Metric | Measured value | Remarks |
|---|---|---|
| Training examples / classes | 80 / 12 | Labelled task and information utterances. |
| Diagnostic requests / correct | 40 / 40 | Local development diagnostic. |
| Outcome accuracy | 100.00% | Final tool or answer outcome. |
| Macro precision | 100.00% | Mean across observed outcome labels. |
| Macro recall | 100.00% | Mean across observed outcome labels. |
| Macro F1-score | 1.0000 | Class-balanced diagnostic summary. |
| Regression suite | 43 passed | Temporary-database service and HTTP checks. |
| Browser workflows | 7 passed | Actual student/admin interface verification. |

## Interpretation and Limits

All 40 diagnostic outcomes matched their expected result in the current build. This result demonstrates those specific authored cases, not 100% general accuracy. The limited vocabulary, small training set and repeated development use can make the diagnostic optimistic. Generalization requires independently labelled student requests, separate validation-only threshold tuning and an untouched test set.

ROC-AUC and binary false-positive/false-negative rates are not reported because the evaluation combines multiple task and answer outcomes rather than a binary calibrated probability model. Similarity scores remain relevance evidence, not a percentage guarantee of correctness.

# CHAPTER 7
# CONCLUSION

## 7.1 Summary of the Project:

NEC Fee Assist has been implemented as an AI-assisted fee portal that connects natural language questions and task requests with authenticated account information and visible portal workflows. Its learned task classifier, TF-IDF retrieval engine, bounded planner and responsive browser interface work together to produce useful answers and controlled actions.

The assistant now goes beyond a text-only question-answer workflow. It can open relevant pages, select a requested fee category, retain task guidance across navigation, continue or cancel a task and find a protected receipt. The student remains responsible for confirming a simulated payment in the normal form.

### Key achievements include:

- A learned multinomial Naive Bayes task model trained from 80 labelled examples across 12 classes.
- Grounded account answers and maintained-policy retrieval with explicit sources and fallback behavior.
- Automatic page navigation and owned pending-fee selection through a fixed interface tool registry.
- Persistent task guidance, session-scoped continuation and cancellation with state rechecks.
- Role and ownership enforcement, independent browser validation and protected receipt access.
- 43 passing regression tests, seven browser workflow checks and a documented 40-request development diagnostic.

## 7.2 Contributions to the Field

The project demonstrates how a small classical AI model can be combined with task tools without granting unrestricted authority to language input. It provides practical laboratory exposure to feature representation, probabilistic learning, retrieval, planning, validation and evidence-based evaluation within an everyday student workflow.

### 7.3 Limitations and Future Work

- Language Coverage: The current model is lexical and English-only. Unfamiliar wording, spelling errors and multi-intent requests can produce an abstention or incorrect task prediction. Larger, independently labelled request datasets are needed.
- Evaluation Independence: The authored diagnostic is reused during development. Future work should separate training, validation and untouched testing, and measure false tool activations as well as correct task completion.
- Semantic Understanding: Semantic embeddings or an optional language model could broaden paraphrase handling. Any replacement must retain the same allow-listed tools, role checks and confirmation boundaries.
- Dialogue Planning: Current task state supports narrow payment, receipt and administrator guides. Richer workflows need explicit progress observations, recovery paths and additional evaluation.
- Production Infrastructure: The loopback Python HTTP server, local SQLite dataset and demonstration accounts are laboratory components. Production use requires approved identity integration, HTTPS, a suitable application server, backups and account lifecycle controls.
- Real Payments: The application simulates payment outcomes. Real gateway orders, provider confirmation, reconciliation and operational controls must be implemented separately before accepting money.

### 7.4 Impact

- Reduces menu searching by connecting a natural language request to its destination page.
- Helps students prepare fee tasks while keeping confirmation visible and controlled.
- Demonstrates how learned intent selection can be combined with grounded data and constrained tools.
- Provides a reproducible educational example of AI-assisted task execution and responsible evaluation.

## 7.5 Final Remarks

The project shows that a fee assistant can help users act as well as answer their questions. By combining a learned task classifier with a bounded planner, authorized account retrieval and direct interface navigation, NEC Fee Assist links conversational intent to practical portal workflows. Its explicit tool set and confirmation boundaries keep the behavior inspectable and its limits clear. Further language evaluation and production integration can build on this working academic foundation.

**RUBRICS :**

| Aim & Algorithm (10 Marks) | Coding & Implementation (20 Marks) | Optimization (10 Marks) | Output (5 Marks) | Time Management (5 Marks) | Viva (10 Marks) | Total (60 Marks) |
|---|---|---|---|---|---|---|
| | | | | | | |

**RESULT:**

NEC Fee Assist has been implemented and locally tested as an AI-based fee assistance and task navigation system. The learned model, grounded answers, automatic page navigation, fee preparation, receipt discovery and guided workflows are operational. All 43 regression tests and seven browser workflow checks passed. The authored 40-request development diagnostic matched all expected outcomes; this is a local diagnostic rather than a general accuracy guarantee. Payments remain simulated and require the normal confirmation form.